

JS JAVASCRIPT MASTERCLASS

DAY 8 OF 30

Objects: Basics, Methods & The `this` Keyword

Understanding key-value data structures, method shorthand, and execution context in modern JavaScript

🕒 30 Minute Session

📄 Live Code & Exercises

01

Object Basics

Key-value structure & 2 creation syntax patterns.

02

Property Access

Dot notation vs bracket notation & dynamic keys.

03

CRUD Operations

Adding, updating, deleting & checking existence.

04

Object Methods

Functions in objects & ES6 shorthand syntax.

05

The `this` Keyword

Execution context & arrow function warnings.

What is an Object?

INDEX vs NAMED KEYS

An object stores data as **key-value pairs** to represent real-world entities with readable property names.

Array (Index-Based)

Unclear Cont

```
let student = ["Ali", 20, "Computer Science"]; console.log(student[0]); // "Ali" – what does index 0 represent
```

Requires remembering positional index order.

Every real-world web entity (user profiles, products, API payloads) is structured as an object.

- ✓ **User Profiles:** Stores user settings & status
- ✓ **API Responses:** Weather, e-commerce, movie data
- ✓ **React State:** Props and state objects in frontend apps

```
// Example: API response for a user profile let userProfile = { name: "Sara Khan", email:
```


</> Creating Objects: 2 Approaches

SYNTAX COMPARISON

1. Object Literal `{}`

99% Standard Usage

```
const car = { brand: "Toyota", model: "Corolla", year: 2023, price: 4500000 }; console.log(car);
```

Clean, readable, and industry standard.

2. Cons

Verbose


```
const student = { name: "Ahmed", age: 21, "favorite subject": "Math" }; // 1. Dot Notation (Standard) console.log(
```


Dynamic Property Access

CRITICAL CONCEPT

```
const student = { name: "Ahmed", age: 21 }; let key = "name"; // Bracket access using variable value console.log(s
```


Add, Update, Delete Properties

MUTATING OBJECTS

```
const laptop = { brand: "Dell", ram: "8GB" }; // 1. ADD a new property laptop.price = 95000; // { brand: "Dell", r
```


🔍 Checking Property Existence

``in` OPERATOR & hasOwnProperty()`

```
const laptop = { brand: "Dell", ram: "16GB" }; // Check using 'in' operator console.log("brand" in laptop); // true
```


Regular Method Syntax

```
const person = { firstName: "Bilal", greet: function () { console.log( "Hello, " + person.firstName ); } }; person.greet();
```



```
const car = { brand: "Toyota", model: "Yaris", year: 2024 }; // 1. Get array of keys Object.keys(car); // ["brand", "model", "year"]
```


CORE RULE

The `this` keyword refers to the object that is **currently executing** the method.

Prevents Code Fragility

If object name changes, methods using `this` won't break.

Dynamic Binding

Value determined at call-time based on invocation context.

`this` Inside Object Methods

BEST PRACTICE

```
const user = { name: "Zainab", email: "zainab@example.com", showInfo: function () { // 'this' refers directly to t
```


⚠ Common Mistake: Global Scope `this`

STANDALONE FUNCTIONS

```
// Standalone function (outside any object) function normalFunction() { console.log(this); } normalFunction(); //
```


⚠ Arrow Function Warning

SIDE-BY-SIDE COMPARISON

Regular / Shorthand Method

✓ Works

```
const demo = { name: "Correct Example", regularMethod: function () { console.log(this.name); } }; demo.regularMethod(); // "Correct Example"
```

ⓘ Rule: Arrow functions do NOT have their own `this`. They inherit `this` from the outer/surrounding scope.

Concept	Key Point
Dot Notation	Used for simple, static property names (<code>obj.key</code>).
Bracket Notation	Required for dynamic keys or keys with spaces (<code>obj[key]</code>).
Object Method	When a property value is a function (<code>greet() {}</code>).
<code>this</code> (Regular Function)	Refers to the object that called the method.
<code>this</code> (Arrow Function)	Has no <code>this</code> — inherits from outer surrounding scope.

Task: `bankAccount` Object

- Create object with `accountHolder` & `balance: 5000`.
- Add `deposit(amount)` method using `this`.
- Add `withdraw(amount)` with "Insufficient funds" check.
- Add `checkBalance()` method.
- Print keys using `Object.keys()`.

```
// Requirements to test: bankAccount.deposit(1000); // Balance should become 6000 bankAccount.withdraw(20000); // Sh
```


✓ Session Summary

5 KEY TAKEAWAYS

- ✓ **Objects:** Store key-value pairs representing real-world entities.
- ✓ **Dot vs Bracket:** Bracket notation is required for dynamic variable keys.
- ✓ **CRUD Operations:** Easily add, update, delete, or check keys with ``in``.
- ✓ **Object Methods:** Prefer ES6 shorthand ``methodName() {}`` syntax.
- ✓ **``this`` Rule:** Use regular function syntax for object methods, not arrow functions.



Keep Coding & Practicing!

Share your solution to the practice assignment in the comments section below!

NEXT SESSION: Day 9 — String Methods: Deep Dive & Practice Problems